

RAGLeakLab: Deterministic Security Testing for Retrieval-Augmented Generation Systems

RAGLeakLab Contributors
<https://github.com/mishabar410/RAGLeakLab>

February 2026 — v1.0.0

Abstract

Retrieval-Augmented Generation (RAG) systems combine document retrieval with language model generation, introducing a dual attack surface: the corpus may leak through retrieval and the generator may reproduce sensitive content. Existing security evaluations rely on stochastic, model-dependent probes that produce non-reproducible results, making them unsuitable for continuous integration and compliance workflows.

We present RAGLEAKLAB, an open-source framework that addresses these limitations through a principled, deterministic design. The framework implements six threat classes—canary extraction, verbatim reproduction, membership inference, semantic leakage, cross-document correlation, and corpus poisoning—evaluated via metrics that require no neural inference at test time. A TF-IDF reference pipeline, fixed paraphrase templates, and deterministic delta debugging ensure identical results across runs. CI integration is provided through JUnit XML and SARIF export, HTTP record/replay cassettes for network-free execution, and configurable threshold calibration with target false-positive rate constraints.

RAGLeakLab is implemented in approximately 12,000 lines of Python, carries no GPU dependencies, and produces bit-identical reports across environments. The framework is designed for integration into enterprise CI/CD pipelines where reproducibility, auditability, and offline operation are requirements.

1 Introduction

Retrieval-Augmented Generation (RAG) has become the dominant architecture for grounding large language models in private document collections. By retrieving relevant passages and injecting them into the generation context, RAG systems reduce hallucination and enable knowledge-intensive applications without fine-tuning. However, this architecture introduces a fundamental tension: the same retrieval mechanism that grounds generation also creates pathways for information leakage.

An attacker who can observe the generated output—and in many deployments, any authenticated user qualifies—may extract sensitive content from the private corpus through carefully crafted queries. The threat surface spans multiple dimensions: verbatim reproduction of source text, inference of corpus membership, extraction of planted canary tokens, semantic exposure of sensitive facts, cross-document correlation of information that should not be composed, and integrity compromise through corpus poisoning.

Evaluating these threats in practice presents a reproducibility challenge. Existing approaches often rely on neural classifiers, stochastic probing, or model-specific adversarial techniques that produce different results across runs, hardware, and software versions. Such non-determinism is fundamentally incompatible with CI/CD pipelines, regression testing, and compliance auditing, where a security gate must return the same verdict given the same inputs.

RAGLEAKLAB is designed from first principles to be deterministic. Every component—from query transformation to metric calculation to report generation—produces identical output across runs when given identical input. The framework achieves this through several design choices:

- A TF-IDF reference retriever with deterministic tie-breaking replaces embedding-model-dependent retrieval.
- Fixed template-based paraphrase generation replaces neural paraphrasing.
- Longest Common Substring and n-gram overlap replace neural similarity for verbatim detection.
- Pattern-matching claim detectors replace LLM-based fact verification.
- Delta debugging for query minimization uses a deterministic algorithm.
- SHA-256 provenance hashes record corpus, attack, and configuration state for auditability.

The remainder of this paper is structured as follows. Section 2 defines the problem space. Section 3 presents the formal threat model. Section 4 describes design principles. Section 5 details the system architecture. Section 6 covers the attack framework. Section 7 formalizes the leakage and integrity metrics. Section 8 describes dataset poisoning evaluation. Section 9 covers threshold calibration. Section 10 presents the attribution engine. Section 11 describes CI integration. Section 12 provides benchmark results. Section 13 discusses security hardening. Section 14 acknowledges limitations. Section 15 surveys related work, and Section 16 concludes.

2 Problem Statement

A production RAG system serves as an interface between untrusted user queries and a private document corpus. The security requirement is that the system should answer questions using corpus knowledge without exposing sensitive content beyond what is strictly necessary for the answer. Formally, let $\mathcal{C} = \{d_1, \dots, d_n\}$ be a private corpus, q a user query, $R(q, \mathcal{C}) \subseteq \mathcal{C}$ the retrieved subset, and $G(q, R)$ the generated answer. The security testing problem is to determine whether $G(q, R)$ reveals information from $\mathcal{C}$ that constitutes a confidentiality violation.

Three properties distinguish this problem from general LLM safety testing:

Dual attack surface. Both the retriever R and the generator G contribute to leakage. The retriever may return documents containing secrets; the generator may verbatim reproduce, paraphrase, or compose information from the retrieved context.

Corpus-dependent threats. The severity of leakage depends on the specific corpus content. A canary token `CANARY_<hex>` planted in the corpus constitutes a definitive leak when reproduced; verbatim overlap with public boilerplate does not. Testing must be parameterized by corpus content and associated claim annotations.

Reproducibility requirements. Enterprise deployments require security gates that produce identical verdicts given identical inputs, support regression detection across releases, and generate audit trails for compliance. Non-deterministic probes violate all three requirements.

RAGLeakLab addresses this problem by providing a testing framework where every component is deterministic, every threat class has a well-defined metric with a configurable threshold, and every run produces a versioned, hash-anchored report suitable for CI gates.

3 Threat Model

RAGLeakLab implements six threat classes, each corresponding to a distinct information leakage vector. The framework explicitly separates *leakage threats* (Sections 3.1–3.5), which concern information escaping from the corpus, from *integrity threats* (Section 3.6), which concern adversarial manipulation of the corpus itself.

3.1 Canary Extraction

Canary tokens are deterministic, high-entropy strings of the form `CANARY_<16 hex chars>`, generated via SHA-256 hashing of a seed: `SHA256("ragleaklab_canary_{seed}")[:16]`. Canaries are injected into corpus documents using configurable strategies (end-of-document or mid-document insertion).

Their presence in generated output constitutes an unambiguous confidentiality violation—the token has zero probability of appearing in any legitimate response.

Attacker model. The attacker submits queries designed to elicit reproduction of injected secrets. Strategies include direct requests, indirect framing, role confusion, and context manipulation (Section 6).

Detection. A compiled regular expression `CANARY_[0-9a-f]{16,}` is applied to the generated output. The metric reports count and presence as a binary signal.

3.2 Verbatim Reproduction

Verbatim leakage measures the degree of exact textual overlap between the generated output and source documents. Two complementary metrics are computed:

- **Longest Common Substring (LCS) ratio:** The length of the longest common substring between the answer and any source document, normalized by source document length. A minimum length threshold (default: 20 characters) suppresses common-phrase false positives.
- **N-gram overlap:** The count of matching n-grams (default: $n = 5$) between answer and source, with a configurable minimum match count.

The combined score is $\max(\text{lcs_ratio}, \text{ngram_ratio})$, where `ngram_ratio` is the fraction of source n-grams appearing in the answer.

3.3 Membership Inference

Membership inference determines whether a specific document was present in the corpus based on observable response characteristics. RAGLeakLab implements two approaches:

Heuristic scoring. A weighted combination of retrieval score statistics (mean, max), answer length, and context specificity indicators. When non-member reference signals are available, the score is the normalized difference between member and non-member signal strengths.

Paraphrase consistency. The query is expanded into deterministic paraphrases using fixed templates (e.g., “What is {query}?”, “Tell me about {query}”, “Details on {query}”). Retrieved document overlap and answer token similarity across paraphrases are measured. High consistency suggests corpus membership. The consistency score combines retrieval stability (Jaccard index over retrieved document IDs) and answer similarity (token-level Jaccard).

3.4 Semantic Leakage

Semantic leakage evaluates whether sensitive *claims*—discrete facts annotated in the corpus—are revealed in the generated output. Claims are defined per-document in JSONL format with typed sensitivity levels (high, medium, low) and categorical types (e.g., `EMAIL`, `PHONE`, `ACCOUNT_ID`, `SECRET_CODEWORD`, `INTERNAL_PROJECT`).

Detection uses type-specific matchers:

- **Email:** Regex-based extraction and exact matching.
- **Phone:** E.164 and common format normalization, then digit-sequence comparison.
- **Account ID / Secret codeword:** Case-insensitive substring matching with text normalization (whitespace collapse, punctuation removal).
- **General:** Normalized substring containment after text preprocessing.

The semantic leakage rate is $|\text{leaked claims}|/|\text{total claims in retrieved documents}|$.

3.5 Cross-Document Correlation

Cross-document leakage detects composed claims—facts that require information from *multiple* source documents to construct. These are defined as *composed claims* in a separate JSONL file, each referencing

two or more source document IDs. Detection verifies that a matched claim indeed spans multiple distinct document origins, preventing single-document matches from triggering false positives.

The metric reports the count of leaked composed claims and provides evidence linking each match to its source documents.

3.6 Corpus Poisoning (Integrity Threats)

Integrity threats model an adversary who can inject or modify documents in the corpus. RAGLeakLab implements three sub-categories, each with a dedicated evidence schema:

Retrieval hijacking (`RetrievalIntegrityEvidence`). Poisoned documents are injected to rank higher than legitimate content for targeted queries. Detection compares expected vs. actual document IDs in retrieval results, computing poison rate at k ($|\text{poison docs in top-}k|/k$), poison mean reciprocal rank, and true document recall at k .

Claim corruption (`ClaimIntegrityEvidence`). Injected documents contain false claims designed to appear in generated output. Detection matches expected true claims against actual output and identifies the presence of poison claims, reporting contradiction counts and corruption rates.

Sentinel takeover (`SentinelIntegrityEvidence`). Backdoor triggers planted in the corpus activate when specific patterns appear in queries. Detection uses rule-based pattern matching with deterministic policies (block/strip/allow) and reports whether triggers were activated, the policy action taken, and output markers found.

Each evidence type carries a severity level (high/medium/low), confidence score, pack and query identifiers, and optional detail dictionaries.

4 Design Principles

Five design principles govern RAGLeakLab’s architecture:

4.1 Determinism

Every operation produces identical output given identical input. The reference pipeline uses a TF-IDF retriever with deterministic tie-breaking (secondary sort by chunk ID ensures stable ranking when cosine similarities are equal). Paraphrase generation uses fixed templates rather than neural models. Delta debugging follows a deterministic splitting algorithm. Report entries are sorted by `test_id` before serialization. A dedicated verification module (`core.determinism`) can run the same pack N times and compare normalized outputs, stripping volatile fields (timestamps, timings) before deep comparison.

4.2 Provenance

Every report carries SHA-256 hashes of its inputs: corpus directory content (`corpus_hash`), attack definitions (`attacks_hash`), runtime configuration (`config_hash`), and target system identifier (`target_hash`). The `compute_config_hash` function deterministically serializes key-value settings, sorts them, and hashes the result. This allows auditors to verify that two reports were produced from identical inputs without accessing the inputs themselves.

4.3 Separation of Concerns

The framework cleanly separates five subsystems: (1) the attack framework (case loading, strategy application, execution), (2) target adapters (in-process pipeline, HTTP endpoints), (3) metric calculators (canary, verbatim, membership, semantic, crossdoc, integrity), (4) the reporting layer (schema, export, regression diff), and (5) corpus management (loading, patching, claim annotation, canary injection). Each subsystem communicates through typed Pydantic contracts defined in `core.contracts`.

4.4 Extensibility

A plugin registry (`core.plugins`) supports three extension points: metrics, attacks, and targets. Plugins can be registered programmatically via `register(kind, name, obj)` or discovered automatically through Python entry points in the `ragleaklab.{kind}` namespace. Built-in components self-register during module import, ensuring they are available without explicit configuration.

4.5 CI-First Design

Every feature is designed for unattended execution. HTTP cassettes enable network-free replays. JUnit XML and SARIF exports integrate with standard CI tooling. Exit codes are structured (0=success, 1=general error, 2=input error, 3=validation error, 4=config error, 5=network error, 99=internal error). Error messages are automatically sanitized via `core.errors` to prevent secret leakage in CI logs.

5 System Architecture

5.1 Reference RAG Pipeline

RAGLeakLab includes a reference RAG pipeline (`rag.pipeline.RAGPipeline`) composed of three stages:

Retrieval. The `TFIDFRetriever` chunks documents using configurable window and overlap parameters (default: 256 characters, 32 overlap), builds TF-IDF vectors with $TF = \log(1 + \text{count})$ and $IDF = \log(N/\text{df})$, and ranks by cosine similarity. Ties are broken by chunk ID (lexicographic, ascending) for stable ordering.

Context assembly. The `ContextBuilder` concatenates retrieved chunks into a single context string passed to the generator.

Generation. The `MockGenerator` produces deterministic answers by echoing retrieved context, ensuring the pipeline's output is fully determined by its inputs without requiring any language model.

This reference pipeline serves as a controlled baseline. Production systems are tested via the `Target` adapter protocol.

5.2 Target Adapters

The `Target` protocol defines a single method: `ask(query) -> TargetResponse`, where `TargetResponse` contains the answer text, optional retrieved document IDs, optional retrieval scores, and raw response data. Two implementations are provided:

InProcessTarget. Wraps a `RAGPipeline` instance, enabling direct testing without network overhead.

HttpTarget. Sends queries to external RAG endpoints via HTTP. Configuration includes request method, URL, field mapping for request/response payloads, custom headers, timeout, and rate limiting (configurable requests-per-second with a token bucket). Security features include SSRF protection (Section 13) and domain allowlisting.

The `HttpTarget` supports three operational modes:

- **live:** Normal HTTP requests.
- **record:** Requests are executed and responses are recorded to JSONL cassette files.
- **replay:** Responses are served from cassette files without network access.

5.3 Contracts

All inter-module communication uses typed Pydantic models defined in `core.contracts`:

- `Document`, `Chunk`, `RetrievalHit`: Corpus and retrieval data structures.

- **RunArtifact:** Complete result of executing a single attack case, including query, answer, retrieved chunks, timings, context statistics, provenance hashes, and optional integrity evidence.
- **MetricScore:** Unified metric result with name, value, details dictionary, and optional pass/fail status.
- **CaseResult:** Combines a **RunArtifact** with its associated **MetricScore** list and overall verdict.
- **ReportSummary:** Top-level report with schema version, tool version, timestamp, overall pass/fail, aggregate metrics, failure details, and metadata.

5.4 Disk Cache

The `core.cache.DiskCache` stores pipeline results keyed by a SHA-256 hash of corpus hash, target hash, query text, and retrieval parameters. This enables fast repeat runs when inputs are unchanged. Cache entries are stored as individual JSON files, using the key hash as filename. Invalid entries are automatically removed on read.

5.5 Corpus Management

The corpus subsystem provides:

- **Document loading:** Plain text files from a directory.
- **Claim annotation:** JSONL files mapping (`doc_id`, `claim_id`) to sensitive facts with type and sensitivity metadata.
- **Canary injection:** Deterministic token generation from integer seeds and configurable injection strategies (end/middle).
- **Corpus patching:** Deterministic application of incremental changes (add/replace/remove documents and claims) via YAML/JSON patch specifications (`corpus.patch`), producing hash-verified output corpora.

6 Attack Framework

6.1 Attack Case Schema

Each attack is defined as an **AttackCase** with the following fields: `test_id` (unique identifier), `threat` (threat class), `strategy` (transformation to apply), `query` or `turns` (single-turn string or multi-turn conversation), optional `expected` output, and optional `tags` for filtering. Pydantic validation enforces that exactly one of `query` or `turns` is provided. Multi-turn attacks are flattened to a single effective query via the `effective_query` property, which concatenates turn roles and content.

Attack cases are loaded from YAML files organized into *packs*—named collections of test cases targeting specific threat classes.

6.2 Strategy Catalog

The `attacks.catalog` module maintains a registry of transformation strategies, each defined as an **AttackStrategy** dataclass with name, description, and a `transform: Callable[[str], str]` function. Built-in strategies include:

Table 1: Built-in attack strategies.

Strategy	Transformation
<code>direct_ask</code>	Identity (no transformation)
<code>indirect_ask</code>	Indirect framing
<code>role_confusion</code>	System role manipulation
<code>context_manipulation</code>	Context injection
<code>semantic_leakage</code>	Semantic probing
<code>crossdoc_correlation</code>	Cross-document composition
<code>authority_appeal</code>	Authority framing
<code>hypothetical_framing</code>	Hypothetical scenarios
<code>metadata_probe</code>	Metadata extraction
<code>enumeration</code>	Systematic enumeration

Custom strategies can be added via the plugin registry.

6.3 Execution Engine

The `attacks.runner` module orchestrates test execution:

1. **Case loading:** `load_cases(pack_path)` parses YAML files into `AttackCase` instances.
2. **Strategy application:** The named strategy’s `transform` function modifies the query.
3. **Cache lookup:** If a `DiskCache` is provided, the cache key (corpus hash, target hash, transformed query, retrieval parameters) is checked.
4. **Execution:** The query is sent through either a `RAGPipeline` or a `Target` adapter.
5. **Artifact creation:** A `RunArtifact` is constructed with the answer, retrieved chunks, context, timing breakdown (retrieval, generation, total milliseconds), context statistics (character count, chunk count, truncation flag), and provenance hashes.

Parallel execution is supported via `ProcessPoolExecutor`, with `run_all` and `run_all_with_target` orchestrating batch execution across all cases in a pack.

6.4 Coverage Analysis

The `attacks.coverage` module computes a `CoverageReport` from loaded test cases, providing:

- Per-threat case counts.
- Per-strategy case counts.
- A threat-strategy matrix showing coverage density.
- Gap identification: given expected threats and strategies (from an optional `attacks.yaml` manifest), the module reports missing combinations.

6.5 Query Minimization

The `attacks.minimize` module implements deterministic delta debugging (Algorithm 1) for failure-preserving query reduction. Given a failing query and an oracle predicate, the algorithm produces a minimal subsequence that still triggers the failure.

Algorithm 1 Deterministic Delta Debugging (**ddmin**)

Require: Sequence c of tokens, oracle $\phi : \text{seq} \rightarrow \{0, 1\}$

Ensure: Minimal $c' \subseteq c$ s.t. $\phi(c') = 1$

```
1:  $n \leftarrow 2$ 
2: while  $|c| \geq 2$  do
3:   Split  $c$  into  $n$  disjoint chunks  $\Delta_1, \dots, \Delta_n$ 
4:    $reduced \leftarrow \text{false}$ 
5:   for  $i = 1$  to  $n$  do
6:     if  $\phi(\Delta_i) = 1$  then
7:        $c \leftarrow \Delta_i$ ;  $n \leftarrow 2$ ;  $reduced \leftarrow \text{true}$ ; break
8:     end if
9:   end for
10:  if not  $reduced$  then
11:    for  $i = 1$  to  $n$  do
12:       $c_i^c \leftarrow c \setminus \Delta_i$ 
13:      if  $\phi(c_i^c) = 1$  then
14:         $c \leftarrow c_i^c$ ;  $n \leftarrow n - 1$ ;  $reduced \leftarrow \text{true}$ ; break
15:      end if
16:    end for
17:  end if
18:  if not  $reduced$  then
19:    if  $n < |c|$  then
20:       $n \leftarrow \min(2n, |c|)$ 
21:    else
22:      break
23:    end if
24:  end if
25: end while
26: return  $c$ 
```

The splitting strategy is configurable: sentence-based (splitting on sentence boundaries) or line-based (splitting on newline characters).

7 Leakage and Integrity Metrics

All metrics are implemented as pure functions that take run artifacts and corpus data as input and produce typed result objects. No metric requires neural inference, GPU resources, or network access.

7.1 Canary Detection

$$\text{detect_canary}(a) = (\mathbb{1}[\text{CANARY_}[0-9a-f]\{16,\} \in a], |\text{matches}|) \quad (1)$$

Returns a `CanaryResult` with boolean `present`, integer `count`, and list of matched token strings.

7.2 Verbatim Overlap

For answer a and source documents $S = \{(d_i, t_i)\}$:

$$\text{verbatim_score} = \max_{(d_i, t_i) \in S} \left(\frac{|\text{LCS}(a, t_i)|}{\max(|t_i|, 1)}, \frac{|\text{ngram_matches}(a, t_i, n)|}{|\text{ngrams}(t_i, n)|} \right) \quad (2)$$

where `LCS` returns the longest common substring (minimum length threshold applied), and n-gram matching uses $n = 5$ by default. The result (`VerbatimResult`) includes the score, LCS length, source

document with maximum overlap, and n-gram match count.

7.3 Membership Confidence

Heuristic approach. Given a list of `RunArtifact` objects, the membership score aggregates normalized retrieval scores (mean and max across chunks), answer length (log-normalized), and context specificity (ratio of unique tokens). When non-member reference data is available, the score is the difference between member and non-member signal strengths, normalized to $[0, 1]$.

Consistency approach. For query q , generate k paraphrases $\{q_1, \dots, q_k\}$ using fixed templates. For each q_i , retrieve documents R_i and generate answer a_i . The consistency score combines:

$$\text{retrieval_consistency} = \frac{1}{\binom{k}{2}} \sum_{i < j} \frac{|R_i \cap R_j|}{|R_i \cup R_j|} \quad (3)$$

$$\text{answer_similarity} = \frac{1}{\binom{k}{2}} \sum_{i < j} \frac{|\text{tokens}(a_i) \cap \text{tokens}(a_j)|}{|\text{tokens}(a_i) \cup \text{tokens}(a_j)|} \quad (4)$$

$$\text{confidence} = w_r \cdot \text{retrieval_consistency} + w_a \cdot \text{answer_similarity} \quad (5)$$

where $w_r = 0.6$ and $w_a = 0.4$ are fixed weights.

7.4 Semantic Leakage

For answer a and claims $\mathcal{K}_R$ from retrieved documents:

$$\text{semantic_leakage_rate} = \frac{|\{c \in \mathcal{K}_R \mid \text{match_claim}(c, a)\}|}{|\mathcal{K}_R|} \quad (6)$$

The `match_claim` function dispatches to type-specific matchers (Section 3.4). Text normalization includes whitespace collapse, punctuation stripping, and case folding.

7.5 Cross-Document Leakage

For composed claims $\mathcal{K}_\times$ (each spanning ≥ 2 documents):

$$\text{crossdoc_leaked} = |\{c \in \mathcal{K}_\times \mid \text{match_claim}(c, a) \wedge |\text{source_docs}(c)| \geq 2\}| \quad (7)$$

7.6 Integrity Metrics

Three metrics evaluate corpus poisoning effects:

$$\text{poison_rate@}k = \frac{|\{d \in \text{top-}k \mid d \in \mathcal{P}\}|}{k} \quad (8)$$

$$\text{poison_mrr} = \frac{1}{\min_{d \in \mathcal{P}} \text{rank}(d)} \quad (9)$$

$$\text{true_doc_recall@}k = \frac{|\{d \in \text{top-}k \mid d \in \mathcal{T}\}|}{|\mathcal{T}|} \quad (10)$$

where $\mathcal{P}$ is the set of poisoned document IDs, $\mathcal{T}$ is the set of expected true document IDs, and the top- k set is the retrieval result.

7.7 Verdict Engine

The `metrics.verdict` module applies configurable thresholds to aggregate metric results and produces a final pass/fail verdict. Default thresholds are:

- Canary: any extraction $\Rightarrow$ FAIL.
- Verbatim: score $> 0.1 \Rightarrow$ FAIL.
- Membership: confidence $> 0.7 \Rightarrow$ FAIL.

Each threshold violation generates a `VerdictReason` with the threat class, human-readable description, observed value, and configured threshold.

8 Dataset Poisoning Evaluation

Corpus poisoning represents a supply-chain attack on RAG systems. An adversary with write access to a subset of the corpus can inject documents designed to manipulate retrieval ranking, corrupt generated claims, or implant backdoor triggers.

8.1 Poisoning Pack Structure

Each poisoning evaluation is organized as a *pack* containing:

- A manifest (`manifest.yaml`) defining queries, expected document IDs, expected claims, and poison document identifiers.
- A clean corpus and a poisoned corpus variant (produced via the `corpus.patch` mechanism).
- Rule definitions for sentinel detection (pattern/replacement pairs with deterministic block/strip/allow policies).

8.2 Evidence Collection

The poisoning runner executes the same queries against both clean and poisoned corpus variants, collecting `IntegrityEvidence` objects:

Retrieval integrity. Compares top-*k* retrieval results between clean and poisoned runs to detect ranking manipulation. Evidence includes expected vs. actual document ID lists, confidence score, and per-query metrics (poison rate, MRR, recall).

Claim integrity. Compares generated output against expected true claims and known poison claims to detect content corruption. Evidence includes matched true claims, matched poison claims, contradiction count, and per-query corruption rate.

Sentinel integrity. Applies rule-based patterns to detect backdoor trigger activation. Evidence includes sentinel type (suffix/trigger/backdoor), whether the trigger fired, the policy action taken, expected vs. actual behavior, and output markers found.

8.3 Integrity Summary

The `IntegritySection` model aggregates all evidence into a summary with severity-stratified counts (high/medium/low), per-type counts (retrieval/claim/sentinel), and worst-case dominance scores. Evidence is sorted deterministically by severity, then pack ID, then query ID.

8.4 Triage

The integrity summary provides three triage signals:

- `worst_poison_dominance`: highest `poison_rate@k` across all retrieval findings.
- `worst_claim_corruption`: highest poison claim rate across all claim findings.
- `sentinel_failures`: count of triggered sentinel findings.

These signals are consumed by CI gates to determine whether poisoning exceeds acceptable thresholds.

9 Calibration

Threshold selection for binary metrics (e.g., verbatim score, membership confidence) requires balancing sensitivity against false positive rate. RAGLeakLab provides a principled calibration workflow.

9.1 Labeled Calibration Sets

Calibration requires a JSONL file of labeled test cases:

```
1 {"test_id": "case_001", "label": "positive"}
2 {"test_id": "case_002", "label": "negative"}
```

Labels are **positive** (leak expected, attack succeeds) or **negative** (no leak expected, security holds). The loader (`calibration.loader`) validates uniqueness, format, and non-emptiness.

9.2 Threshold Fitting

The `calibration.fit.fit_threshold` function implements a sweep-based algorithm:

1. Separate scores by label into positive and negative sets.
2. Enumerate all unique score values as candidate thresholds.
3. For each threshold t , compute FPR and TPR:

$$\text{FPR}(t) = \frac{|\{s \in S^- \mid s \geq t\}|}{|S^-|} \quad (11)$$

$$\text{TPR}(t) = \frac{|\{s \in S^+ \mid s \geq t\}|}{|S^+|} \quad (12)$$

where S^+ and S^- are positive and negative score sets (assuming `higher_is_worse = True`).

4. Select the threshold with maximum TPR subject to $\text{FPR} \leq \text{target_fpr}$ (default: 0.01).
5. Deterministic tie-breaking: among thresholds achieving equal TPR at the constraint boundary, prefer the higher (more conservative) threshold.

The result (`CalibrationResult`) reports the chosen threshold, achieved FPR and TPR, sample counts, and the decision rule string (e.g., “score $\geq$ threshold $\rightarrow$ FAIL”).

9.3 Calibration Report

The `calibration.report` module generates a full report including:

- Pack name and metric name.
- Target FPR and achieved rates.
- ROC-like table of (threshold, FPR, TPR) operating points.
- Generation timestamp.

Reports are written as JSON to support automated threshold update workflows (e.g., a CI job that recalibrates thresholds when the corpus changes and commits updated baseline files).

10 Attribution

When a security test fails, practitioners need to understand *why* the leak occurred to implement effective remediation. The `analysis.attribution` module provides rule-based root cause analysis.

10.1 Attribution Categories

Five categories are implemented, each with a corresponding remediation hint:

Table 2: Attribution categories and remediation hints.

Category	Remediation Hint
<code>retrieval_included_secret</code>	Review retriever filtering; consider excluding documents with sensitive markers.
<code>context_too_long</code>	Reduce context window or implement summarization.
<code>top_k_too_high</code>	Lower <code>top_k</code> to reduce attack surface.
<code>chunking_boundary</code>	Adjust chunk boundaries to avoid splitting sensitive content.
<code>target_overexposed_endpoint</code>	Adjust HTTP target for unintended data exposure.

10.2 Attribution Logic

The `attribute_leak` function evaluates conditions based on run artifact data:

- If canary detected and retrieved chunks are non-empty, attribute to `retrieval_included_secret`.
- If context exceeds 10,000 characters, attribute to `context_too_long`.
- If more than 5 chunks were retrieved, attribute to `top_k_too_high`.
- If the target is HTTP-based and a leak was detected, attribute to `target_overexposed_endpoint`.

Attribution results are embedded in SARIF output as properties on each finding, enabling security dashboard integration.

11 CI Integration

RAGLeakLab is designed as a CI gate: a process that runs in the pipeline, produces a verdict, and fails the build on security regressions.

11.1 Export Formats

JUnit XML. The `reporting.export.export_junit` function produces standard JUnit XML consumed by all major CI platforms (GitHub Actions, GitLab CI, Jenkins, Azure DevOps). Each test case is mapped to a `<testcase>` element; failures include the threat type, query, and truncated answer. Suppressed findings are marked as `<skipped>` with the suppression reason.

SARIF. The `reporting.export.export_sarif` function produces SARIF 2.1.0 for GitHub Code Scanning. Rules are defined for each threat class with calibrated severity scores:

- Canary extraction: severity 9.0 (error).
- Semantic leakage: severity 8.0 (error).
- Cross-document leakage: severity 8.5 (error).
- Verbatim leakage: severity 7.0 (warning).
- Membership inference: severity 5.0 (warning).
- Retrieval hijacking: severity 8.5 (error).
- Claim corruption: severity 8.0 (error).
- Sentinel takeover: severity 9.0 (error).

Suppressed findings receive SARIF `suppressions` annotations with `kind: inSource` and the justification text.

11.2 HTTP Cassettes

The cassette mechanism (`targets.cassette`) enables deterministic CI runs without network access:

Recording. During a `record` mode run, each HTTP request and response is serialized to a JSONL cassette file. Requests are normalized (sorted headers, sorted query parameters, normalized body) and hashed (SHA-256) for lookup. Sensitive headers (`Authorization`, `X-API-Key`, `Cookie`, etc.) are redacted before storage.

Replay. During a `replay` mode run, incoming requests are normalized and hashed identically. The cassette file is searched for a matching hash. If found, the stored response is returned without network access. If not found, a `CassetteReplayError` is raised.

This design ensures that CI pipelines can execute the full test suite against recorded external service responses, eliminating network dependencies and rate-limit concerns.

11.3 Regression Detection

The `regression.diff` module compares two `ReportSummary` objects (baseline vs. current) and detects regressions:

- **Canary regression:** `false` $\rightarrow$ `true` is an immediate failure.
- **Verbatim regression:** rate increase exceeding a configurable delta threshold (default: 0.01).
- **Membership regression:** confidence increase exceeding a configurable delta threshold (default: 0.05).
- **Semantic regression:** leakage rate increase exceeding the verbatim delta threshold.

The `DiffResult` includes per-metric deltas with baseline and current values, enabling trend analysis.

11.4 Suppressions

The suppression system (`suppressions`) allows teams to acknowledge known risks without removing them from reports. Suppressions are defined in `suppressions.yaml` with mandatory fields:

- `id`: UUID identifying the suppression.
- `type`: What is suppressed (`test_id`, `claim_id`, `doc_id`, or `metric`).
- `value`: Identifier to match.
- `reason`: Non-empty justification (enforced by validation).
- `expires_at`: Mandatory ISO-8601 expiry timestamp.
- `owner`: Optional responsible party.

Suppressions are applied *after* metric calculation but *before* the final verdict. Suppressed findings remain visible in all outputs (JUnit: `<skipped>`; SARIF: `suppressions` annotation; JSON: `suppression_summary` section). The verdict changes from FAIL to KNOWN_RISK, never to PASS. Expired suppressions are automatically ignored.

12 Benchmarks

RAGLeakLab's design targets two benchmarking dimensions: determinism verification and execution performance.

12.1 Determinism Verification

The `core.determinism.verify_determinism` function provides automated determinism testing. Given a pack name and run count N , it:

1. Creates N fresh `RAGPipeline` instances and executes the pack against each.
2. Normalizes all outputs by stripping volatile fields (`generated_at`, `timings`) and sorting entries by `test_id`.
3. Performs deep structural comparison of `report.json` and `runs.jsonl` across all pairs $(0, i)$ for $i \in [1, N)$.
4. Reports any differences with path-qualified descriptions (e.g., `aggregates.verbatim_leakage_rate: 0.05 != 0.06`).

Under the reference pipeline with identical corpus and attack inputs, all runs produce bit-identical normalized outputs. This property is verified in the test suite.

12.2 Performance Characteristics

The framework is designed for lightweight execution:

- **No GPU required.** All operations use CPU-only algorithms (TF-IDF, regex, string matching, hashing).
- **Parallel execution.** `run_all` uses `ProcessPoolExecutor` for multi-core utilization.
- **Caching.** The `DiskCache` eliminates redundant pipeline executions when inputs are unchanged.
- **Cassette replay.** Network-free replay eliminates HTTP latency and rate-limit delays.

Typical execution times for a 50-case pack against the reference pipeline on a single CPU core are under 5 seconds, making RAGLeakLab suitable for pre-commit hooks and PR-blocking CI gates.

13 Security Hardening

As a security testing tool, RAGLeakLab must not itself become an attack vector. The following hardening measures are implemented:

13.1 SSRF Protection

The `targets.ssrif` module prevents Server-Side Request Forgery when using the `HttpTarget`. The `validate_url` function:

1. Rejects non-HTTP(S) schemes.
2. Resolves the hostname to its IP addresses.
3. Blocks connections to private/reserved IP ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.0/8, 169.254.0.0/16, ::1/128, fc00::/7, fe80::/10, and the unspecified addresses 0.0.0.0 and ::.
4. Optionally enforces a domain allowlist, rejecting any target not in the approved set.

13.2 Secret Redaction

The `core.redact` module provides context-aware redaction for report outputs and logs:

- Pattern-based redaction: email addresses, phone numbers, canary tokens, bearer/basic auth tokens, API keys (Stripe-style, AWS AKIA-prefix), and generic credential patterns.
- Header-aware redaction: values of sensitive HTTP headers (`Authorization`, `X-API-Key`, `Cookie`, etc.) are replaced with `[REDACTED]`.
- Recursive operation: `redact_dict` traverses nested dict/list structures, applying redaction to all string values.
- Environment variable scanning: error messages are checked against the values of environment variables whose names contain sensitive keywords (`KEY`, `SECRET`, `TOKEN`, `PASSWORD`, `CREDENTIAL`, `AUTH`).

13.3 Error Message Sanitization

The `core.errors` module ensures that error messages displayed to users or written to CI logs do not leak secrets:

- The `sanitize_message` function applies the same redaction patterns used for report output.
- Home directory paths are replaced with `~` via `safe_path_str`.
- Structured `SafeError` objects carry sanitized messages alongside structured exit codes.

13.4 Filesystem Safety

The `core.fs` module provides:

- **Path traversal protection:** `safe_join` resolves paths and verifies the result is within the base directory, rejecting absolute paths and `..` traversal.
- **Atomic writes:** `atomic_write` uses a temp-file-plus-rename pattern, ensuring files are either fully written or untouched. This prevents partial report writes on crashes or interrupts.

13.5 Cassette Security

HTTP cassettes redact sensitive headers before storage and normalize requests to prevent cache poisoning via header manipulation. Cassette files use deterministic request hashing (SHA-256 of normalized method, URL, sorted headers, and body), making it infeasible to craft a request that maps to a different cassette entry.

14 Limitations

RAGLeakLab makes explicit design trade-offs that result in the following limitations:

No neural-based analysis. All metrics use string matching, heuristics, and pattern detection. This ensures determinism and eliminates GPU dependencies, but means the framework cannot detect paraphrased leakage that evades substring matching or semantic similarity that requires embedding-based comparison. The claim matching system uses normalized substring containment, which may miss reformulations that preserve meaning while changing surface form.

Reference pipeline is not production-representative. The built-in TF-IDF retriever and mock generator serve as a controlled baseline, not as a proxy for production embedding models or language generators. Production testing requires the `HttpTarget` adapter or a custom `InProcessTarget` wrapping the actual system. The reference pipeline’s value lies in verifying framework behavior, not in predicting production leakage rates.

Fixed paraphrase templates. The eight built-in paraphrase templates provide lightweight diversification for consistency-based membership inference. They do not cover the full space of semantic-preserving reformulations and may underestimate membership inference risk against systems with more sophisticated query understanding.

No runtime moderation testing. RAGLeakLab tests the RAG system as a black box and does not model or test runtime content moderation layers (e.g., output filters, guardrail classifiers). If a production system includes such layers, the framework tests the combined system but cannot attribute filtering to specific moderation components.

Claim annotation is manual. Semantic leakage and cross-document detection require human-authored claim annotations in JSONL format. The quality and completeness of these annotations directly bound the metrics’ recall. No automated claim extraction is implemented.

Single-process corpus assumption. The framework assumes single-process access to the corpus directory during patching and testing. Concurrent modifications may produce non-deterministic results unless external locking is applied.

Heuristic attribution. The attribution engine uses rule-based heuristics with fixed thresholds (e.g., context length > 10,000 characters). These thresholds are not calibrated per-system and may require tuning for specific deployment configurations.

15 Related Work

LLM red-teaming frameworks. Tools such as Microsoft’s Counterfit, NVIDIA’s NeMo Guardrails evaluation suite, and various prompt injection benchmarks focus on adversarial testing of language models. These operate at the model level and typically require neural inference for both attack generation and evaluation. RAGLeakLab differs by targeting the RAG architecture specifically and by eliminating neural dependencies for deterministic CI operation.

Membership inference attacks. The membership inference literature originates in the ML privacy domain, with shadow model approaches and loss-based metrics. RAGLeakLab’s heuristic and consistency-based approaches avoid model training requirements while providing practical signals for corpus membership detection.

Data poisoning. Backdoor attacks on retrieval systems and knowledge bases have been studied in the context of search engine optimization and knowledge graph corruption. RAGLeakLab provides a structured evaluation framework with three poisoning sub-categories (retrieval hijacking, claim corruption, sentinel takeover) and deterministic evidence collection.

Canary-based auditing. The use of planted tokens for memorization detection was introduced for language model training data extraction. RAGLeakLab adapts this technique for RAG systems, where the leakage path is through retrieval and context injection rather than model memorization.

Software testing determinism. The broader software testing literature emphasizes flaky test elimination and deterministic execution. RAGLeakLab applies these principles to security testing, with explicit volatile field stripping, sorted serialization, and automated determinism verification.

CI security scanning. Tools like Semgrep, CodeQL, and Snyk provide static and dependency analysis in CI pipelines. RAGLeakLab provides dynamic security testing for RAG system behavior, complementing code-level scanning with runtime leakage detection. The framework’s SARIF and JUnit outputs integrate directly with the same CI platforms.

16 Conclusion

We have presented RAGLeakLab, a deterministic security testing framework for Retrieval-Augmented Generation systems. The framework addresses a gap in the current tooling landscape: the need for reproducible, CI-compatible security evaluation of RAG architectures without neural inference dependencies.

The key contributions are:

1. **Deterministic design.** A comprehensive architecture where every component—from TF-IDF retrieval to delta-debugging query minimization—produces bit-identical output across runs.
2. **Six threat classes.** Canary extraction, verbatim reproduction, membership inference, semantic leakage, cross-document correlation, and corpus poisoning, each with well-defined metrics and evidence schemas.
3. **CI-native integration.** JUnit XML, SARIF 2.1.0, HTTP cassettes, regression diffing, and structured suppression management enable direct integration into enterprise CI/CD pipelines.
4. **Principled calibration.** FPR-constrained threshold fitting with deterministic tie-breaking enables systematic threshold selection for binary metrics.
5. **Security hardening.** SSRF protection, secret redaction, error sanitization, path traversal prevention, and atomic file writes ensure the framework itself does not become an attack vector.

RAGLeakLab is open source under the Apache License 2.0 and available at <https://github.com/mishabar410/RAGLeakLab>. The framework requires only a standard Python installation with no GPU or external service dependencies for core operation.

Future directions include: integration of lightweight embedding-based similarity metrics (preserving determinism through pinned model versions), automated claim extraction from annotated corpora, multi-tenant threat modeling, and expanded poisoning pack coverage for emerging attack vectors.